



Consumo de APIs con React



**Activen las cámaras los que puedan y
pasemos asistencia**

***Construir aplicaciones web
en react para mostrar
información dinámica
proveniente de una base de
datos de terceros.***

- Unidad 1: Introducción a React.
- Unidad 2: Estados de los componentes y eventos.
- Unidad 3: Renderización dinámica de componentes.
- Unidad 4: Consumo de APIs con React.
- Unidad 5: React Router I
- Unidad 6: Context
- Unidad 7: React Router II
- Unidad 8: JWT



Te encuentras aquí





Inicio

{desafío}
latam_



*/** Construir aplicaciones web en React que consuman APIs. **/*

**Objetivo de
la unidad**

/ Utilizar useEffect para modificar el título de la página. */*

/ Utilizar useEffect exclusivamente al momento del montaje. */*

/ Conocer el ciclo de vida de los componentes. */*

/ Utilizar useEffect en conjunto con fetch para consumir una API. */*

/ Utilizar useRef para referenciar y modificar elementos del DOM. */*

Objetivos

Encontremos el error

Cuando retornamos información a partir de un arreglo de elementos:

¿Por qué se genera este error al momento de recorrer dichos elementos para mostrarlos en el HTML.?

```
⚠ Warning: Each child in a list should have a unique "key" prop.  
  
Check the render method of `App`. See https://reactjs.org/link/warning-keys for more information.  
    at li  
    at App
```




Desarrollo

{desafío}
latam_



Introducción a useEffect

useEffect

¿Para qué sirve useEffect?

useEffect es una función que permite que un componente pueda ejecutar alguna acción después de haber sido renderizado, por ejemplo:

1. Leer alguna base de datos, conectarse a una API y extraer la información de ella.
2. Actualizar el DOM, cambiando el título del sitio a partir de ciertos cambios o acciones que se ejecuten.
3. Escribir en una base de datos, conectarse a una API para hacer POST de información.
4. Todas aquellas acciones que puedan cambiar el resultado de nuestros componentes.

useEffect

useEffect es un hook que controla efectos secundarios

- Este tipo de acciones, que no se hacen durante el renderizado del componente, reciben el nombre de **Side Effects** o efectos secundarios.
- Es por eso que la función se llama useEffect, y al igual que useState es un hook.

useEffect

Introducción a useEffect

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Similar to componentDidMount and componentDidUpdate:
  useEffect(() => {
    // Update the document title using the browser API
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

useEffect

Introducción a useEffect

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Similar to componentDidMount and componentDidUpdate:
  useEffect(() => {
    // Update the document title using the browser API
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

1. Para utilizar useEffect tenemos que importarlo.
2. useEffect recibe un callback con la función que se ejecutará. En este caso, la función cambia el título del documento.
3. La función pasada se llamará automáticamente cuando el componente se monte o actualice.

Ejercicio guiado

Realiza los siguientes pasos, utilizando el ejemplo de la imagen anterior:

1. Crea un proyecto desde cero llamado **ejemplo-efecto**.
2. Reemplaza el componente App.jsx por el código del ejemplo de la imagen anterior. Puedes copiar directamente el código en el link de la [documentación oficial](#).



Recuerda renombrar la función a App.

Recuerda agregar el export default al final del archivo.

3. Abre el proyecto con el navegador.
4. Observa el cambio del título del tab cuando se hace clic en el documento.
5. Abre el inspector de elementos en el tab componente.
 - a. Hace clic en App.
 - b. Observa con la herramienta react-developer-tools los cambios en el state y en el effect.

{desafío}
latam_

Demostración



/ Utilizar useEffect para modificar el título de la página. */*



/ Utilizar useEffect exclusivamente al momento del montaje. */*

/ Conocer el ciclo de vida de los componentes. */*

/ Utilizar useEffect en conjunto con fetch para consumir una API. */*

/ Utilizar useRef para referenciar y modificar elementos del DOM. */*

Objetivos

**`/* useEffect y el ciclo de vida de un
componente */`**

Ciclo de vida

Las fases de un componente

Los componentes tienen 3 fases:

Montaje

El componente es cargado la primera vez y mostrado en el DOM.

Actualización

El componente recibe alguna actualización de su estado inicial.

Desmontaje

Etapas donde el componente se remueve el DOM.

Ejercicio guiado

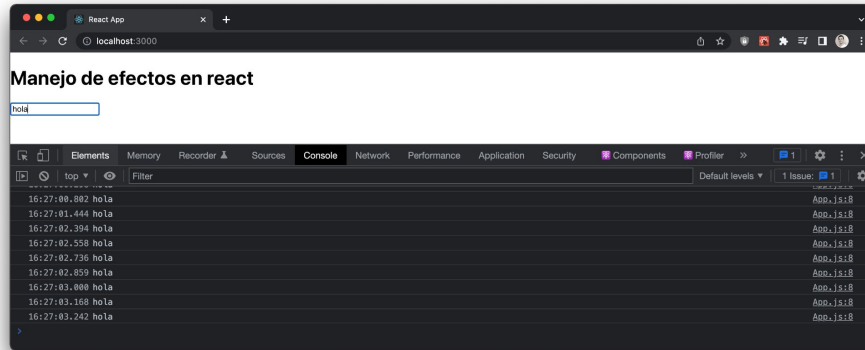
Para ver en la práctica la importancia del ciclo de vida, crearemos una aplicación llamada ejemplo-efecto. En esta generaremos un input que guarde cada cambio en un estado. Adicionalmente, agregaremos `useEffect` con un mensaje del tipo "hola mundo" con `console.log`.

- Cuando se cargue el componente, se mostrará el mensaje en el inspector de elementos, demostrando que `useEffect` se gatilla en el montaje del componente.
- Cuando se agregue o borre un caracter del input, veremos que el mensaje se muestra en el inspector de elementos, demostrando que `useEffect` también se gatilla al momento de actualizar el componente.

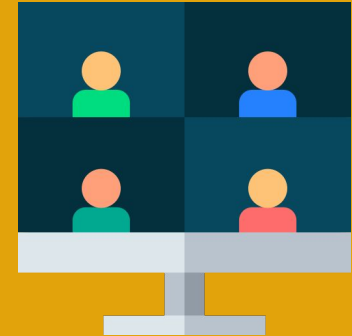
Demostración



1. Importamos useState y useEffect.
2. Utilizamos useState para guardar el estado de un input, llamémosle nombre y al setter setNombre con valor inicial vacío.
3. Utilicemos useEffect para mostrar un mensaje hola en pantalla.
4. Agreguemos el input que escuche el evento onChange.
5. Observemos que el mensaje se muestra tanto al montar el componente (recargar la página para este propósito) así como cambiar un texto del input.



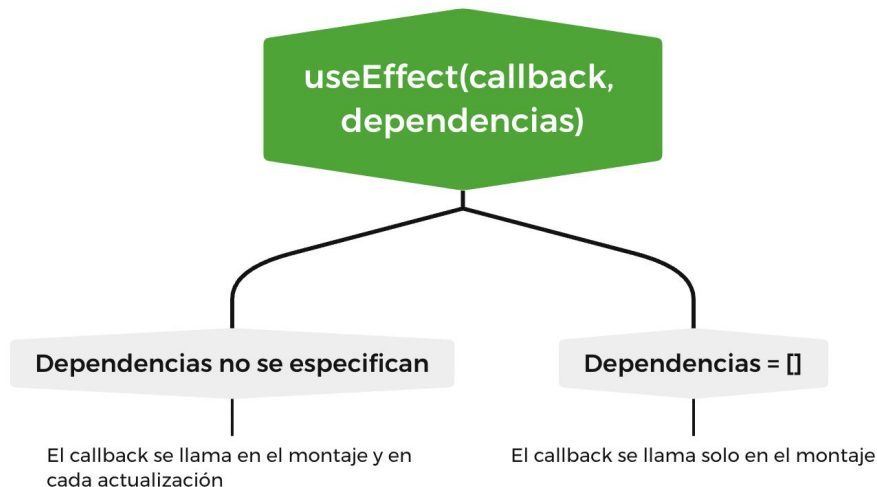
Demostración



useEffect

Arreglo de dependencias

Para evitar que useEffect se gatille en cada actualización, se le puede pasar un segundo argumento adicional al callback llamado arreglo de dependencias.



useEffect

Pregunta: ¿Cuál es el callback?

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // Similar to componentDidMount and componentDidUpdate:
  useEffect(() => {
    // Update the document title using the browser API
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

useEffect

Limitando el efecto solo a la fase de montaje

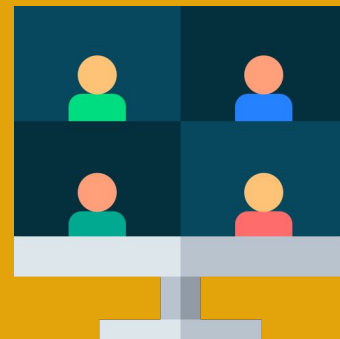
```
useEffect(() => {  
  console.log("hola");  
}, []);
```

- Solo tenemos que agregar un arreglo vacío después del callback.

Ejercicio

1. Modifiquemos el proyecto que acabamos de crear para que el efecto solo se llame al cargar el componente.
2. Probemos que haya funcionado en el inspector de elementos.
3. Lo único extraño que veremos, es que el mensaje se muestra dos veces, esto se debe a que en el modo estricto de React los componentes se renderizan dos veces para poder detectar fallos.

Demostración



Ejercicio

Repite los pasos

1. Importa useState y useEffect.
2. Utiliza useState para guardar el estado.
3. Utiliza useEffect para mostrar un mensaje con console.log.
4. Agrega un input de texto que al hacer un cambio en él se modifique el estado.
5. Agrega un arreglo vacío de dependencias para evitar que useEffect se gatille tras cada cambio.

Ejercicio

¡Manos al teclado!



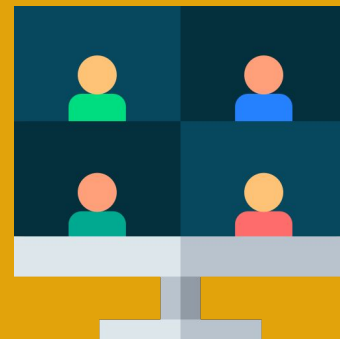
Ejercicio

1. Realicemos un último experimento, en `useEffect` especifiquemos el estado dentro del input dentro del arreglo de dependencias. Si el estado es nombre, entonces:

```
useEffect(() => console.log("hola"), [nombre])
```

Aquí veremos que nuestro código vuelve a funcionar de la misma forma que inicialmente, puesto que lo que estamos indicando es que el efecto depende de la actualización de nombre, y este es el único estado.

Demostración



useEffect(callback, dependencias)

Dependencias no se especifican

— El callback se llama en el montaje y en cada actualización

Dependencias = []

— El callback se llama solo en el montaje

Dependencias = [nombre, email]

— El callback se llama en la fase de montaje y en la fase de actualización pero solo cuando nombre o email cambian

/ Utilizar useEffect para modificar el título de la página. */*



/ Utilizar useEffect exclusivamente al momento del montaje. */*



/ Conocer el ciclo de vida de los componentes. */*



/ Utilizar useEffect en conjunto con fetch para consumir una API. */*

/ Utilizar useRef para referenciar y modificar elementos del DOM. */*

Objetivos

Consumo de información de una API

useEffect y las API

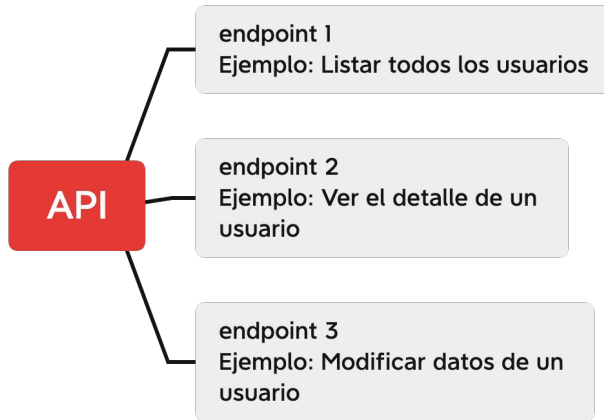


- Un uso interesante de `useEffect` es consumir APIs.
- Antes de aprender cómo hacerlo, revisemos algunos conceptos importantes.

Consumo de información de una API

Repaso Endpoints

- Dentro de una API puede haber uno o más endpoints, así como dentro de un sitio web pueden haber múltiples páginas web. Cada endpoint provee un servicio distinto y tiene una URL distinta.
- Un endpoint se compone de una acción + una URL.
- La acción y URL de cada endpoint junto a lo que devuelve, se especifica en la documentación de cada API.



Consumo de información de una API

Repaso: Haciendo un request con fetch

Para consumir APIs utilizaremos fetch.

Aunque también es posible utilizar Axios.

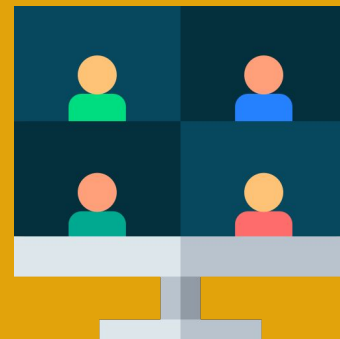
```
async function getRandomUser(){  
  const res = await fetch("https://randomuser.me/api")  
  const data = await res.json()  
  console.log(data);  
}  
  
getRandomUser()
```

Ejercicio guiado

Para crear un componente que muestre datos consumidos de una API necesitamos:

1. Una función que nos devuelva los resultados de la API.
2. Llamar a la función creada desde `useEffect`.
3. Un estado, donde guardaremos los valores que nos devuelve la API.
4. Utilizar el estado en el return de la página para mostrar los datos.

Demostración




```
import { useState, useEffect } from "react";
function App() {
  // 3. info guardará los valores traídos desde la API
  const [info, setInfo] = useState([]);

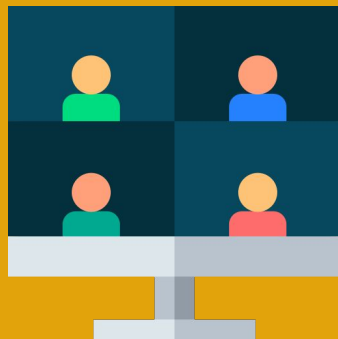
  // 2. Llamamos al función que consume la API al momento de montar el
  componente
  useEffect(() => {
    consultarInformacion();
  }, []);

  // 1. Función que consulta la API
  const consultarInformacion = async () => {
    const url = 'https://api.gameofthronesquotes.xyz/v1/random';
    const response = await fetch(url)
    const data = await response.json()
    setInfo(`${data.sentence}` - `${data.character.name}`); // con setInfo
    actualizamos el estado
  }

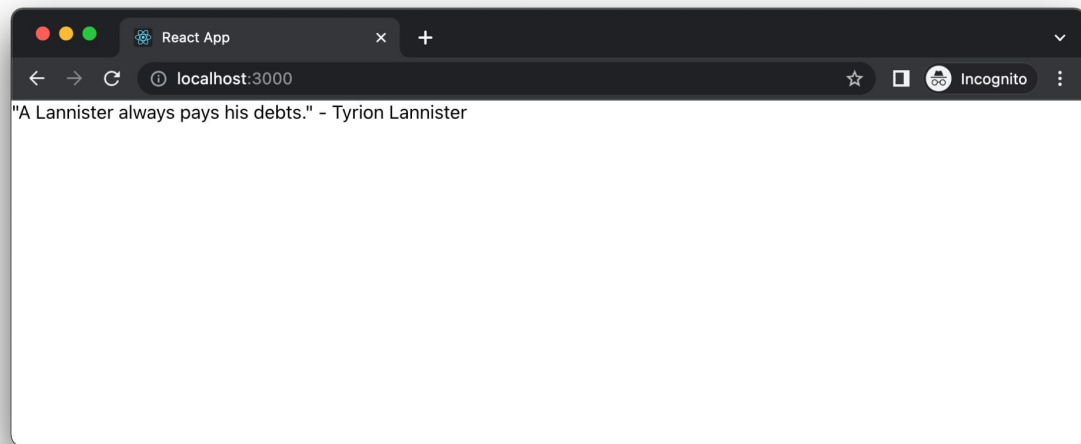
  return (
    <div>
      {/* { 4. Mostramos la info } */}
      { info }
    </div>
  );
}

export default App;
```

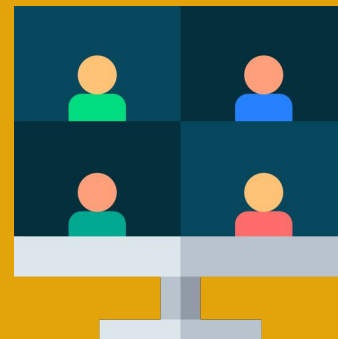
Demostración



Al cargar la aplicación, veremos una frase de la serie de Game Of Thrones:



Demostración



Resultado de ejecución

useEffect

Desde usuarios

- Leanne Graham
- Ervin Howell
- Clementine Bauch
- Patricia Lebsack
- Chelsey Dietrich
- Mrs. Dennis Schulist
- Kurtis Weissnat
- Nicholas Runolfsdottir V
- Glenna Reichert
- Clementina DuBuque

En esta actividad utilizaremos lo aprendido para consultar la API de usuarios de jsonplaceholder.

Endpoint: get

<https://jsonplaceholder.typicode.com/users>

Donde se mostrarán solo los nombres.

Ejercicio

Pasos importantes.

1. Construir una función que nos devuelva los resultados de la API.
2. Llamar a la función creada desde useEffect.
3. Agregar un estado, donde guardaremos los valores que nos devuelve la API.
4. Utilizar el estado en el return de la página para mostrar los datos.

Puedes utilizar los pasos de la guía como base.

Ejercicio

¡Manos al teclado!



/ Utilizar useEffect para modificar el título de la página. */*



/ Utilizar useEffect exclusivamente al momento del montaje. */*



/ Conocer el ciclo de vida de los componentes. */*



/ Utilizar useEffect en conjunto con fetch para consumir una API. */*



/ Utilizar useRef para referenciar y modificar elementos del DOM. */*

Objetivos

`/* Conozcamos useRef */`

useRef

¿Para qué sirve useRef?

useRef permite almacenar una referencia a algún elemento del DOM.

En otras palabras, permitirá acceder y modificar elementos del DOM directamente, almacenar valores mutables a lo largo del ciclo de vida del componente y evitar renderizaciones innecesarias.

Con useRef podemos:

1. Acceder a elementos del DOM.
2. Mantener valores mutables.
3. Animaciones o interacciones personalizadas.
4. Optimizar el rendimiento.

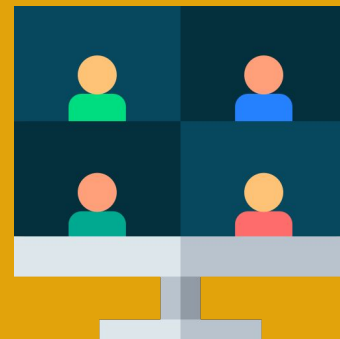
Ejercicio guiado

Para ver en la práctica el uso de useRef crearemos un proyecto llamado ejemplo-ref.

En esta generaremos dos botones que se encargarán de abrir y cerrar un dropdown, y también un div que tendrá el contenido del dropdown.

Abrir Dropdown	Cerrar Dropdown
Contenido del Dropdown	

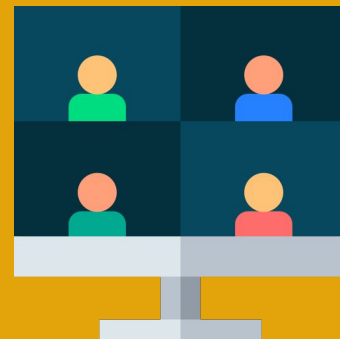
Demostración



Ejercicio guiado

1. Crearemos una referencia a nuestro div mediante:
const dropdownRef = useRef()
Esta referencia nos servirá para acceder al elemento del DOM
2. Generamos dos funciones, una para abrir el dropdown y otra para cerrarlo.
3. Dentro de estas funciones accedemos al elemento referenciado con dropdownRef.current y le aplicamos el estilo necesario para ocultar y mostrar el contenido del dropdown.

Demostración

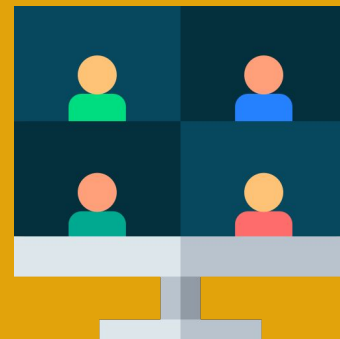


Ejercicio guiado

4. En cada botón, se ejecutará el evento onClick para llamar a cada función.
5. Por último, en el div contenedor del dropdown, utilizaremos el atributo ref para utilizar la referencia que creamos.

Con estos pasos podremos abrir y cerrar nuestro dropdown.

Demostración



/ Utilizar useEffect para modificar el título de la página. */*



/ Utilizar useEffect exclusivamente al momento del montaje. */*



/ Conocer el ciclo de vida de los componentes. */*



/ Utilizar useEffect en conjunto con fetch para consumir una API. */*



/ Utilizar useRef para referenciar y modificar elementos del DOM. */*



Objetivos

Cerrando la unidad

Conceptos clave

1. ¿En qué situaciones necesitamos ocupar useEffect?
2. ¿Cuáles son las fases de un componente?
3. ¿Para qué sirve el callback que le pasamos a useEffect?
4. ¿Para qué sirve el arreglo de dependencias que recibe useEffect?
5. ¿Para qué sirve el hook useRef?



Cierre

{desafío}
latam_



- Revisemos el material asincrónico de la unidad.
 - Se revisará el concepto de DOM virtual.
 - Adicionalmente aparecen preguntas que pueden aparecer en una entrevista laboral.
- Revisemos la prueba del módulo.

¿Qué sigue?

¿Tienen alguna duda respecto a la prueba?



*Academia de
talentos digitales*

www.desafiolatam.com



/DesafioLatam



/DesafioLatam



/DesafioLatam



/DesafioLatam